



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: _Aaron Moises Orellana Aguirre_ **Carné:** _____1220025_____

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Un estilo de arquitectura que permite que dos sistemas se comuniquen por HTTP, usando recursos identificados por URLs y operaciones estándar.

2. ¿Qué es un endpoint?

Una dirección específica de la API donde se puede acceder a un recurso o función

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección web completa. El endpoint es la ruta concreta de la API que ofrece un recurso, forma parte de la URL y la combinación con un método HTTP define la acción.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

- **GET:** obtiene datos.
- **POST:** crea un recurso nuevo.
- **PUT:** reemplaza un recurso completo.
- **PATCH:** modifica solo una parte del recurso.
- **DELETE:** elimina un recurso.

5. ¿Qué es JSON y por qué es común en las APIs?

Es un formato de texto ligero para representar datos con pares clave-valor. Es común porque es fácil de leer y compatible con casi todos los lenguajes.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Número que indica el resultado de una solicitud.

- **200:** OK, la solicitud fue exitosa.
- **404:** Not Found, el recurso no existe.
- **500:** Internal Server Error, falló algo en el servidor.

7. ¿Qué información se puede encontrar en la respuesta de una API?

Código de estado, encabezados, cuerpo con los datos solicitados o un mensaje de error.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Permite probar endpoints sin escribir código, ver las respuestas al instante y así detectar errores temprano

9. ¿Qué es y para qué sirve Swagger UI?

Herramienta que genera documentación interactiva de una API a partir de su especificación. Sirve para ver los endpoints disponibles y probarlos desde el navegador.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Cumple la función de enviar solicitudes a APIs, organizarlas en colecciones, automatizar pruebas y documentar el comportamiento de la API



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>
API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
GET/pet/{petId}
- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
500
- Explique con sus palabras qué información contiene la respuesta.
Error, hubo un error procesando mi petición

Evidencia: captura de Swagger mostrando la operación y su respuesta.

Request URL

```
https://petstore3.swagger.io/api/v3/pet/12200
```

Server response

Code	Details
------	---------

500	Error: response status is 500
-----	-------------------------------

Response body

```
{  
  "code": 500,  
  "message": "There was an error processing your request. It has been logged (ID: 7a622480a8037540)"  
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization  
access-control-allow-methods: GET,POST,DELETE,PUT  
access-control-allow-origin: *  
access-control-expose-headers: Content-Disposition  
content-length: 110  
content-type: application/json  
date: Wed, 30 Sep 2026 13:43:11 GMT  
server: Jetty(9.4.53.v20231009)
```



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 12200,
  "name": "Niebla",
  "category": {
    "id": 2,
    "name": "Gato"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "string"
    }
  ],
  "status": "available"
}
```



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
 - GET – Consultar mascota por ID
 - POST – Crear mascota
 - PUT – Modificar mascota
 - GET – Buscar por status
 - DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.



- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.